

System Design

I. System Architecture

The system follows a monolithic architecture where the backend is built as a single unified application handling all core business logic, data access, and services in one deployable unit. The frontend is developed separately using React and communicates with the backend through exposed RESTful APIs.

Components include the backend (controllers, services, repositories, and database layer) and the React frontend (UI, state management, and API integration layer). Communication happens over HTTP using JSON payloads.

This structure was chosen for simplicity, easier development and deployment, and faster debugging since all backend modules are centralized in one codebase while keeping a clear separation from the frontend.

II. Data Model

The system database schema consists of Users, Applications, Documents, Audit Logs, and Application Actions.

- Users store authentication details, roles, and profile information.
- Applications represent the core business entity submitted by users, linked to a user (submitted by) and their institution and tracking status changes.
- Documents are attached to applications and store file metadata such as type, path, and upload time.
- Audit Logs record all critical system activities like creation, updates, approvals, and rejections for traceability.
- Application Actions track every user interaction with an application (e.g., review, edit, comment), and help enforce rules such as preventing a user from approving an application they previously worked on.

The relationships are mainly one-to-many, where a user can have many applications, actions, documents, and audit logs. This structure ensures accountability, traceability, and strong control over the application workflow.

III. State Machine

The application lifecycle is managed using a finite state machine that enforces strict control over all transitions based on state, user role, and action. The main

states are DRAFT, SUBMITTED, UNDER_REVIEW, INFO_REQUESTED, RESUBMITTED, REVIEWED, VERIFIED, APPROVED, and REJECTED, with APPROVED and REJECTED as terminal states.

Each transition is only allowed if a valid combination of current state + action + role is satisfied; otherwise, an exception is thrown to prevent invalid workflow progression.

The system enforces strict valid state transitions based on the current state, user role, and performed action to ensure a controlled workflow. An application moves from DRAFT to SUBMITTED when an applicant submits it, and from INFO_REQUESTED to RESUBMITTED when additional information is provided. Reviewers can transition an application from SUBMITTED or RESUBMITTED to UNDER_REVIEW, and within this state they may move it to INFO_REQUESTED, REVIEWED, or REJECTED depending on the action taken. Verifiers then process REVIEWED applications into VERIFIED, request more information, or reject them, while approvers can move VERIFIED applications to either APPROVED or REJECTED. Once an application reaches APPROVED or REJECTED, it becomes a terminal state with no further transitions allowed.

The system also enforces a key governance rule where a user cannot approve an application they previously worked on, ensuring separation of duties. Overall, this design guarantees a controlled, auditable, and rule-driven application process without skipping or repeating stages.

IV. Roles

The system defines four main roles: Applicant, Reviewer, Verifier, and Approver, each with clearly separated responsibilities to enforce a structured and secure workflow.

The Applicant can create applications, submit them, and resubmit when information is requested, but cannot review, verify, or approve any application to ensure separation from decision-making.

The Reviewer is responsible for examining submitted applications, requesting additional information, completing reviews, or rejecting at the review stage, but cannot verify or give final approval.

The Verifier ensures compliance and correctness after review, with the ability to verify, request more information, or reject, but cannot approve final decisions.

The Approver has the final authority to approve or reject applications after verification but cannot modify earlier stages or perform reviews.

These boundaries were designed to enforce segregation of duties, reduce bias and conflict of interest, and ensure that no single user can influence an application from start to final approval.

V. Hard decisions

Here are the hard design decisions and trade-offs, aligned with your system requirements, and how your implementation addresses them:

We made a deliberate choice to enforce a strict state machine instead of a flexible workflow model, ensuring that every transition is explicitly validated through (state + action + role) checks. This satisfies the non-negotiable requirement of no invalid or skipped states, and guarantees controlled progression, though it reduces flexibility and requires more upfront logic definition.

For role-based access control (RBAC), we hard-coded separation between Applicant, Reviewer, Verifier, and Approver inside the state machine. This ensures strong segregation of duties, preventing any single user from controlling the full lifecycle. The trade-off is reduced configurability, as role rules are embedded in logic rather than externalized.

To enforce auditability and traceability, we introduced Application Actions and Audit Logs that record every decision and transition. This satisfies the requirement for full lifecycle tracking, but increases database writes and schema complexity.

Finally, for the conflict-of-interest rule (no self-approval), we explicitly check historical actions before allowing approval. This ensures integrity but adds runtime validation overhead and dependency on historical data consistency.

With more time, we would improve the UI/UX to better visualize role-based workflows and enhance usability through dashboards, workflow tracking, and clearer user feedback. We would also refactor the system to use a configurable state machine, a policy-based role/permission engine (RBAC/ABAC), and a visual workflow engine. Additionally, we would optimize audit and action storage for scalability and analytics. Overall, the current design prioritizes correctness, governance, and auditability over flexibility and performance optimization.